Analisis de los resultados

Nombre: Alejandra Del Rosario Ramos Mamani
Código: 2024-119036
Ejecución del ejemplo

```
alejandra@alejandra-VirtualBox:~/Escritorio/05_pasoMensajes$ ./recv
Are you hearing me?
```

```
alejandra@alejandra-VirtualBox:~/Escritorio/05_pasoMensajes$ ./send
```

1. **Interacción solicitada:**
   En la terminal A

```
alejandra@alejandra-VirtualBox:~/Escritorio/05_pasoMensajes$ ./p1
Process 1 sent: Are you hearing me?
Process 1 received: Loud and clear
Process 1 sent: I can hear you too
```

   En la terminal B

```
alejandra@alejandra-VirtualBox:~/Escritorio/05_pasoMensajes$ ./p2
Process 2 received: Are you hearing me?
Process 2 sent: Loud and clear
Process 2 received: I can hear you too
```

   Código de p1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#define MSGSZ 128

typedef struct msgbuf {
    long mtype;
    char mtext[MSGSZ];
} message_buf;

int main() {
    int msqid;
    key_t key = 1234;
    message_buf sbuf, rbuf;

    // Crear la cola de mensajes
    if ((msqid = msgget(key, IPC_CREAT | 0666)) < 0) {
        perror("msgget P1");
        exit(1);
    }
    // 1. Envia "Are you hearing me?" (mtype = 1)
    sbuf.mtype = 1;
    strcpy(sbuf.mtext, "Are you hearing me?");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd P1 (mtype 1)");
        exit(1);
    }
    printf("Process 1 sent: %s\n", sbuf.mtext);
    // 2. Recibe la respuesta (mtype = 2) - Bloqueante
    if (msgrcv(msqid, &rbuf, MSGSZ, 2, 0) < 0) {
        perror("msgrcv P1 (mtype 2)");
        exit(1);
    }
    printf("Process 1 received: %s\n", rbuf.mtext);
    // 3. Replica "I can hear you too" (mtype = 3)
    sbuf.mtype = 3;
    strcpy(sbuf.mtext, "I can hear you too");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd P1 (mtype 3)");
        exit(1);
    }
    printf("Process 1 sent: %s\n", sbuf.mtext);
    return 0;
}
```

Código de p2

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#define MSGSZ 128
typedef struct msgbuf {
    long mtype;
    char mtext[MSGSZ];
} message_buf;

int main() {
    int msqid;
    key_t key = 1234;
    message_buf rbuf, sbuf;

    if ((msqid = msgget(key, 0666)) < 0) {
        perror("msgget P2");
        exit(1);
    }
    // 1. Recibe el primer mensaje (mtype = 1)
    if (msgrcv(msqid, &rbuf, MSGSZ, 1, 0) < 0) {
        perror("msgrcv P2 (mtype 1)");
        exit(1);
    }
    printf("Process 2 received: %s\n", rbuf.mtext);
    // 2. Responde "Loud and clear" (mtype = 2)
    sbuf.mtype = 2;
    strcpy(sbuf.mtext, "Loud and clear");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd P2 (mtype 2)");
        exit(1);
    }
    printf("Process 2 sent: %s\n", sbuf.mtext);
    // 3. Recibe la réplica (mtype = 3)
    if (msgrcv(msqid, &rbuf, MSGSZ, 3, 0) < 0) {
        perror("msgrcv P2 (mtype 3)");
        exit(1);
    }
    printf("Process 2 received: %s\n", rbuf.mtext);
    // Limpieza de la cola del kernel
    msgctl(msqid, IPC_RMID, NULL);

    return 0;
}
```

2. **Servidor y dos clientes en una sola cola (comunicación privada)**
   Código de server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>

#define MSGSZ 128
#define MSG_TO_SERVER 100 // Tipo común para recibir peticiones

typedef struct {
    long mtype;
    int client_id;
    char mtext[MSGSZ];
} client_msg;

int main() {
    int msqid;
    key_t key = 5678; // Clave única para el servidor
    client_msg msg;

    // Crear cola de mensajes IPC
    if ((msqid = msgget(key, IPC_CREAT | 0666)) < 0) {
        perror("msgget server");
        exit(1);
    }

    printf("Servidor iniciado. Esperando mensajes...\n");

    // Procesar peticiones de 2 clientes
    for (int i = 0; i < 2; i++) {
        if (msgrcv(msqid, &msg, sizeof(client_msg) - sizeof(long), MSG_TO_SERVER, 0) < 0) {
            perror("msgrcv server");
            exit(1);
        }

        printf("[Servidor] Recibido de Cliente %d: '%s'\n", msg.client_id, msg.mtext);
```

```
        // Preparar respuesta privada para ese cliente especifico
        int dest_client = msg.client_id;
        msg.mtype = dest_client;

        if (dest_client == 1) {
            strcpy(msg.mtext, "Hola Cliente 1, tu solicitud fue procesada.");
        } else if (dest_client == 2) {
            strcpy(msg.mtext, "Hola Cliente 2, tu solicitud fue procesada.");
        }

        // Enviar respuesta privada
        if (msgsnd(msqid, &msg, sizeof(client_msg) - sizeof(long), 0) < 0) {
            perror("msgsnd server");
            exit(1);
        }
        printf("[Servidor] Respuesta privada enviada a Cliente %d.\n", dest_client);
    }

    printf("Servidor finalizó de responder a ambos clientes.\n");
    return 0;
}
```

Código de cliente1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#define MSGSZ 128
#define MSG_IO_SERVER 100
#define CLIENT_ID 1

typedef struct {
    long mtype;
    int client_id;
    char mtext[MSGSZ];
} client_msg;

int main() {
    int msqid;
    key_t key = 5678;
    client_msg msg;

    if ((msqid = msgget(key, 0666)) < 0) {
        perror("msgget client1");
        exit(1);
    }
    // 1. Enviar mensaje al servidor
    msg.mtype = MSG_IO_SERVER;
    msg.client_id = CLIENT_ID;
    strcpy(msg.mtext, "Petición del Cliente 1");

    if (msgsnd(msqid, &msg, sizeof(client_msg) - sizeof(long), 0) < 0) {
        perror("msgsnd client1");
        exit(1);
    }
    printf("[Cliente 1] Mensaje enviado al servidor.\n");
    // 2. Leer solo la respuesta privada enviada a mtype = CLIENT_ID (1)
    if (msgrcv(msqid, &msg, sizeof(client_msg) - sizeof(long), CLIENT_ID, 0) < 0) {
        perror("msgrcv client1");
        exit(1);
    }

    printf("[Cliente 1] Respuesta recibida: '%s'\n", msg.mtext);
    return 0;
}
```

Código de cliente2.c

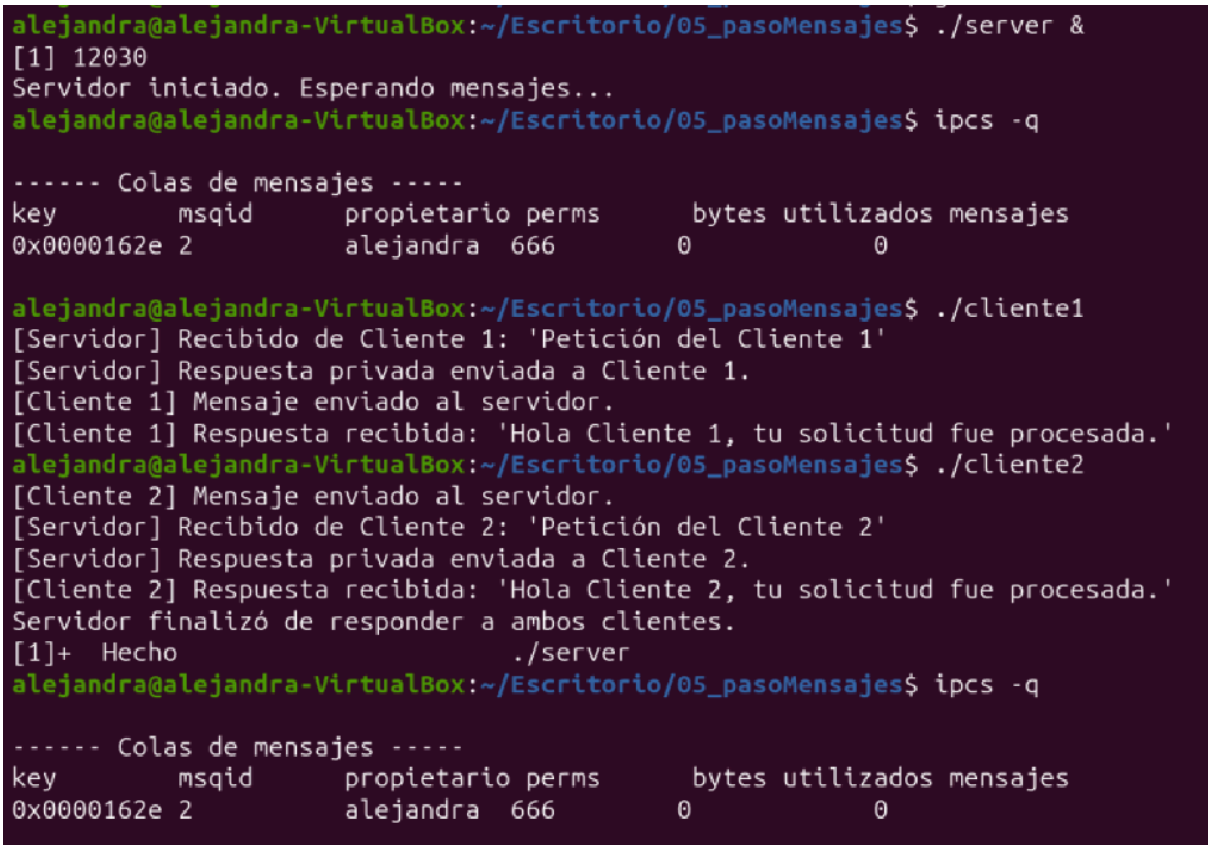
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#define MSGSZ 128
#define MSG_IO_SERVER 100
#define CLIENT_ID 2

typedef struct {
    long mtype;
    int client_id;
    char mtext[MSGSZ];
} client_msg;

int main() {
    int msqid;
    key_t key = 5678;
    client_msg msg;

    if ((msqid = msgget(key, 0666)) < 0) {
        perror("msgget client2");
        exit(1);
    }
    // 1. Enviar mensaje al servidor
    msg.mtype = MSG_IO_SERVER;
    msg.client_id = CLIENT_ID;
    strcpy(msg.mtext, "Petición del Cliente 2");
    if (msgsnd(msqid, &msg, sizeof(client_msg) - sizeof(long), 0) < 0) {
        perror("msgsnd client2");
        exit(1);
    }
    printf("[Cliente 2] Mensaje enviado al servidor.\n");
    // 2. Leer SOLAMENTE la respuesta privada enviada a mtype = CLIENT_ID (2)
    if (msgrcv(msqid, &msg, sizeof(client_msg) - sizeof(long), CLIENT_ID, 0) < 0) {
        perror("msgrcv client2");
        exit(1);
    }
    printf("[Cliente 2] Respuesta recibida: '%s'\n", msg.mtext);
    return 0;
}
```

Ejecución



Explicación del Esquema Utilizado

Para multiplexar la comunicación entre un servidor y múltiples clientes dentro de una sola cola IPC sin que se crucen los mensajes:

- Filtro mediante mtype (Mapeo de Tipos de Mensajes):
    - Peticiones al Servidor: Todos los clientes envían sus peticiones usando el tipo de mensaje mtype = 100.
    - Respuestas Privadas:
        - Cada cliente tiene asignado un identificador único.
        - En el cuerpo del mensaje del cliente, este incluye su ID (clientID).
        - Cuando el servidor responde, establece mtype = clientID.
    - Filtrado del cliente:
        - Cada cliente llama a msgrcv() especificando únicamente su propio mtype.
        - Por ejemplo, el client1 solicitará únicamente mensajes con mtype = 1 y el client2 con mtype = 2. De este modo, los mensajes de un cliente no son leídos por el otro.